SPIEGAZIONE SEGUENDO LO SCRIPT

Sentence_split_nltk.py

```
import os
# importo il modulo os, che serve per interagire con il sistema operativo:
# permette di gestire cartelle, leggere file, costruire percorsi e salvare output

import nltk
# importo la libreria nltk, una libreria di Natural Language Processing
# che contiene strumenti linguistici già pronti, tra cui il sentence tokenizer

from nltk.tokenize import sent_tokenize
# da nltk.tokenize importo direttamente la funzione sent_tokenize,
# che serve a dividere automaticamente un testo in frasi


nltk.download("punkt")
# scarico la risorsa "punkt", cioè il modello che NLTK usa per riconoscere
# i confini di frase. Questo modello permette di segmentare il testo
# sfruttando la punteggiatura e regole statistiche

nltk.download("punkt_tab")
# scarico anche "punkt_tab", una risorsa di supporto che in alcuni ambienti
# è richiesta per il corretto funzionamento del tokenizer di NLTK


cartella_input = "data/raw"
# definisco la cartella di input, cioè la cartella in cui si trovano i file
# testuali originali da elaborare

cartella_output = "output/sentence_split_nltk"
# definisco la cartella di output, cioè la cartella in cui verranno salvati
# i file contenenti le frasi segmentate con NLTK


if not os.path.exists(cartella_output):
    os.makedirs(cartella_output)
# controllo se la cartella di output esiste già.
# os.path.exists restituisce True se esiste, False se non esiste.
# con not inverto il risultato, quindi entro nell'if solo se la cartella manca.
# os.makedirs crea materialmente la cartella di output


lista_file = os.listdir(cartella_input)
# con os.listdir leggo tutti gli elementi presenti nella cartella di input
# e li salvo nella variabile lista_file sotto forma di lista


for nome_file in lista_file:
    if nome_file.endswith(".sent_split"):
# avvio un ciclo for che scorre tutti i file presenti nella cartella di input
# la variabile nome_file assume ogni volta il nome del file corrente
```

```
# con endswith(".sent_split") filtro solo i file che terminano con questa estensione,
# ignorando eventuali altri file non utili al task

    percorso_input = os.path.join(cartella_input, nome_file)
    percorso_output = os.path.join(cartella_output, nome_file)
# costruisco il percorso completo del file di input e quello del file di output
# os.path.join unisce correttamente cartella e nome del file
# adattando automaticamente il separatore al sistema operativo

    file_input = open(percorso_input, "r", encoding="utf-8")
# apro il file di input in modalità lettura ("r") con codifica UTF-8,
# utile per leggere correttamente caratteri accentati e testi multilingui

    testo = file_input.read()
# leggo tutto il contenuto del file in una sola volta
# e lo salvo nella variabile testo come una stringa

    file_input.close()
# chiudo il file dopo la lettura, così libero le risorse usate dal sistema

    testo = testo.replace("<EOS>", " ")
# sostituisco tutte le occorrenze del token speciale <EOS> con uno spazio
# serve a ripulire il testo prima della segmentazione, eliminando marcatori artificiali

    if nome_file.startswith("it_"):
      lingua = "italian"
    else:
      lingua = "english"
# qui scelgo automaticamente la lingua del tokenizer
# se il nome del file inizia con "it_" considero il testo italiano
# e quindi imposto lingua = "italian"
# altrimenti considero il testo inglese e imposto lingua = "english"
# questa scelta è importante perché sent_tokenize usa modelli dipendenti dalla lingua

    frasi = sent_tokenize(testo, language=lingua)
# applico la funzione sent_tokenize al testo
# NLTK segmenta il contenuto in una lista di frasi
# usando il modello della lingua scelta poco sopra
# il risultato viene salvato nella variabile frasi

    frasi_finali = []
# creo una lista vuota che conterrà le frasi finali pulite

    for frase in frasi:
      frase = frase.strip()
      if frase != "":
        frasi_finali.append(frase)
# ciclo su tutte le frasi prodotte da sent_tokenize
# con strip rimuovo eventuali spazi iniziali, finali o newline
# con if frase != "" controllo che la frase non sia vuota
# se è valida la aggiungo alla lista frasi_finali con append

    file_output = open(percorso_output, "w", encoding="utf-8")
# apro il file di output in modalità scrittura ("w")
# se il file non esiste viene creato, se esiste viene sovrascritto
```

```
    for frase in frasi_finali:
        file_output.write(frase + "\n")
# ciclo su tutte le frasi finali e le scrivo nel file di output
# aggiungo "\n" dopo ogni frase così nel file finale avrò una frase per riga

    file_output.close()
# chiudo il file di output dopo aver completato la scrittura

    print("Creato file NLTK:", nome_file)
# stampo a schermo un messaggio che indica quale file è stato creato


print("Ho finito.")
# stampo un messaggio finale che indica la conclusione del programma
```

Sentence_split_spaCy.py

```
import os
# importo il modulo os, necessario per gestire file, cartelle e percorsi

import spacy
# importo la libreria spaCy, una libreria di NLP che permette di analizzare
# linguisticamente un testo e, tra le varie cose, segmentarlo in frasi


cartella_input = "data/raw"
# definisco la cartella in cui si trovano i file di input da elaborare

cartella_output = "output/sentence_split_spacy"
# definisco la cartella di output in cui verranno salvati i file segmentati


if not os.path.exists(cartella_output):
    os.makedirs(cartella_output)
# controllo se la cartella di output esiste già
# se non esiste, la creo con os.makedirs per evitare errori in scrittura


modello_inglese = spacy.load("en_core_web_sm")
# carico il modello linguistico inglese di spaCy
# questo modello contiene le regole e i componenti necessari per elaborare testi inglesi

modello_italiano = spacy.load("it_core_news_sm")
# carico il modello linguistico italiano di spaCy
# servirà per segmentare correttamente i testi italiani

modello_inglese.max_length = 2000000
modello_italiano.max_length = 2000000
# qui aumento il valore massimo di lunghezza del testo accettato dai modelli spaCy
# di default spaCy impone un limite oltre il quale può andare in errore
# aumentando max_length rendo il programma più tollerante verso testi lunghi
```

```
lista_file = os.listdir(cartella_input)
# ottengo la lista di tutti gli elementi presenti nella cartella di input


def dividi_in_blocchi(testo, grandezza_blocco=500000):
# definisco una funzione personalizzata chiamata dividi_in_blocchi
# riceve come parametri il testo intero e la dimensione massima desiderata per ogni blocco
# lo scopo è spezzare testi troppo lunghi in parti più piccole prima di passarli a spaCy

  blocchi = []
# creo una lista vuota che conterrà tutti i blocchi di testo

  inizio = 0
# imposto una variabile inizio a 0
# questa indicherà la posizione da cui partire per estrarre il prossimo blocco

  while inizio < len(testo):
# avvio un ciclo while che continua finché non ho raggiunto la fine del testo
# len(testo) restituisce la lunghezza totale della stringa

    fine = inizio + grandezza_blocco
# definisco un punto finale teorico del blocco,
# sommando alla posizione iniziale la dimensione massima desiderata

    if fine >= len(testo):
      blocchi.append(testo[inizio:])
      break
# se il punto finale supera o raggiunge la fine del testo,
# allora prendo tutto il testo rimanente da inizio fino alla fine
# lo aggiungo alla lista blocchi e interrompo il ciclo con break

    pezzo = testo[inizio:fine]
# estraggo una porzione del testo dalla posizione inizio alla posizione fine

    ultimo_spazio = pezzo.rfind(" ")
# cerco l'ultimo spazio presente nel pezzo estratto
# rfind restituisce la posizione dell'ultima occorrenza trovata,
# oppure -1 se non trova nulla

    if ultimo_spazio != -1:
      fine = inizio + ultimo_spazio
# se trovo almeno uno spazio, sposto la fine del blocco su quell'ultimo spazio
# così evito, quando possibile, di spezzare una parola a metà

    blocchi.append(testo[inizio:fine])
# aggiungo alla lista blocchi il segmento definitivo trovato

    inizio = fine
# aggiorno la posizione iniziale per il blocco successivo,
# facendola partire dal punto in cui si è fermato il blocco corrente

  return blocchi
# restituisco la lista completa dei blocchi creati
```

```
for nome_file in lista_file:
  if nome_file.endswith(".sent_split"):
# ciclo su tutti i file della cartella di input
# considero solo quelli che terminano con ".sent_split"

    percorso_input = os.path.join(cartella_input, nome_file)
    percorso_output = os.path.join(cartella_output, nome_file)
# costruisco i percorsi completi del file di input e del file di output

    file_input = open(percorso_input, "r", encoding="utf-8")
# apro il file di input in modalità lettura con codifica UTF-8

    testo = file_input.read()
# leggo l'intero contenuto del file e lo salvo nella variabile testo

    file_input.close()
# chiudo il file dopo la lettura

    testo = testo.replace("<EOS>", " ")
# sostituisco il token speciale <EOS> con uno spazio,
# così ripulisco il testo da marcatori artificiali

    if nome_file.startswith("it_"):
      modello = modello_italiano
    else:
      modello = modello_inglese
# scelgo automaticamente quale modello spaCy usare
# se il file inizia per "it_" uso il modello italiano
# altrimenti uso il modello inglese

    blocchi = dividi_in_blocchi(testo, 500000)
# applico la funzione personalizzata dividi_in_blocchi al testo
# così ottengo una lista di segmenti più piccoli da elaborare separatamente

    frasi_finali = []
# creo una lista vuota che conterrà tutte le frasi finali estratte dai vari blocchi

    for blocco in blocchi:
      doc = modello(blocco)
# ciclo su ogni blocco di testo
# passando il blocco al modello spaCy ottengo un oggetto doc,
# cioè una rappresentazione linguistica del testo elaborato

      for frase in doc.sents:
        frase_pulita = frase.text.strip()
        if frase_pulita != "":
          frasi_finali.append(frase_pulita)
# doc.sents restituisce le frasi riconosciute da spaCy nel blocco corrente
# per ogni frase estraggo il testo con frase.text
# applico strip per eliminare spazi inutili
# controllo che la stringa non sia vuota
# poi la aggiungo a frasi_finali con append

    file_output = open(percorso_output, "w", encoding="utf-8")
```

```
# apro il file di output in modalità scrittura con codifica UTF-8

    for frase in frasi_finali:
        file_output.write(frase + "\n")
# scrivo tutte le frasi finali nel file di output
# aggiungendo una newline dopo ciascuna,
# così ottengo una frase per riga

    file_output.close()
# chiudo il file di output dopo aver terminato la scrittura

    print("Creato file spaCy:", nome_file)
# stampo a schermo il nome del file appena creato


print("Ho finito.")
# stampo il messaggio finale di chiusura del programma
```